

Machine Learning

Week 1 Lecture

Dr. Muhammad Ahmed Raza

Tentative Plan

- Before Mid:
 - Solidify core mathematics for ML (Algorithms/optimization)
- After Mid:
 - Understand MLOps (i.e., pipelines, deployment, etc.)
- Assessment Plan:
 - Quizzes (30 marks):
 - Viva: if possible (will be graded as a separate quiz)
 - Quiz: every week (probably in the lab's first 10 mins) - online
 - Assignments (10 marks):
 - 4 assignments

LMS

- Google Classroom
 - Details will be disseminated by the CR
 - Course outline will be shared on GCR
- Custom Webapp for quizzes:
 - Needs some testing (any volunteers?)
 - Details will be shared via google classroom announcement in the 1st week

Expectations

- I expect you to know:
 - Basics of ML/DL algorithms (e.g., supervised, semi-supervised, methodologies, etc.)
 - Python (Classes/Methods), Basic Matrix Multiplication, and what a "Feature" is.
- I do NOT expect you to love:
 - Summation notation ($\sum$) or Greek-heavy proofs.

From API Consumer to System Architect

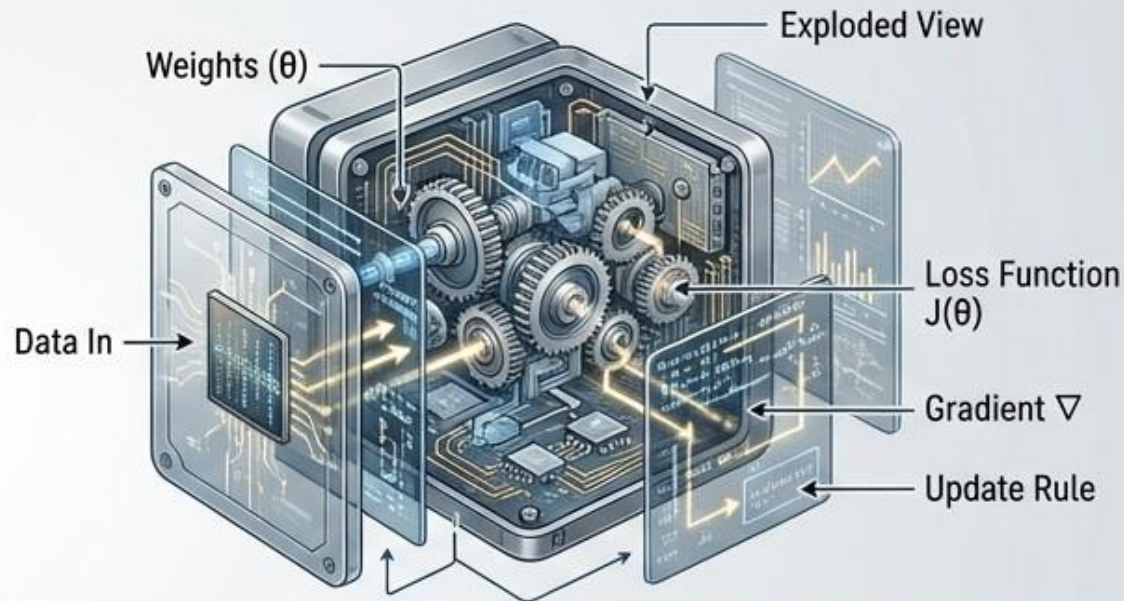
The Past: SE310T

```
model = LinearRegression()  
model.fit(X_train, y_train)
```

Black Box Abstraction

- User Mentality
- Library dependence (sklearn)
- Focus on application

The Present: SE4xx



The Machinery

- Builder Mentality
- Low-level primitives
- Focus on mathematical mechanics

Foundations (Wk 1-2)

Algorithms from Scratch (Wk 3-5)

Production Systems (Wk 10-15)

Machine Learning

Deconstructing the Black Box

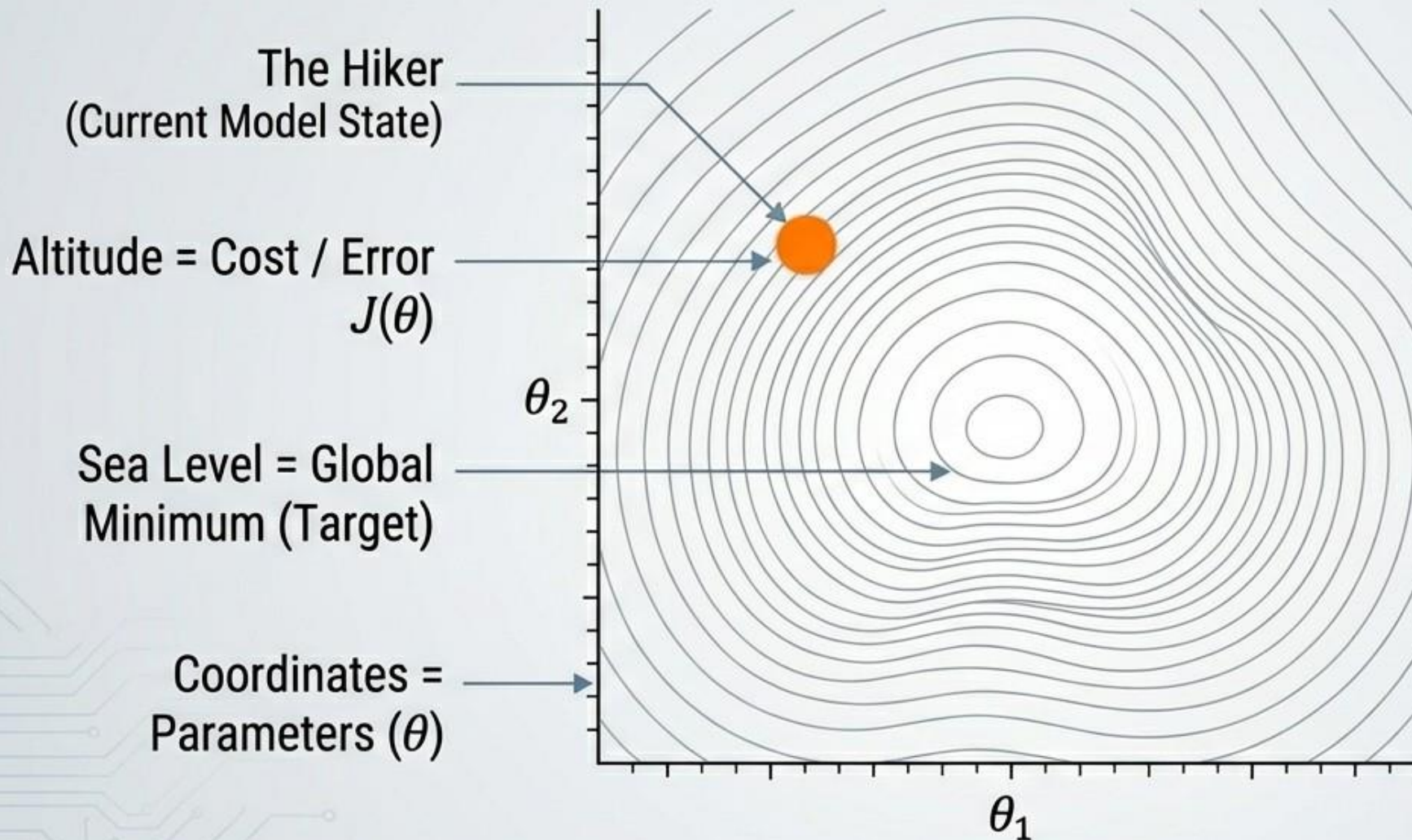
`.fit()`



$\operatorname{argmin} J(\theta)$

- Machine Learning is not magic; it is **Optimization**.
- **The Goal:** Find the specific set of parameters (Weights & Biases, θ) that minimizes error.
- **The Tool:** A metric called the **Loss Function** $J(\theta)$.
- **The Method:** We do not 'guess' the answer. We slide down the error surface to find it.

The Objective: Minimize $J(\theta)$



The Blindfold Problem:

The algorithm cannot see the full map. It acts like a blindfolded hiker.

It can only feel the slope of the ground immediately beneath its feet to decide which way to step.

Foundations (Wk 1-2)

Algorithms from Scratch (Wk 3-5)

Production Systems (Wk 10-15)

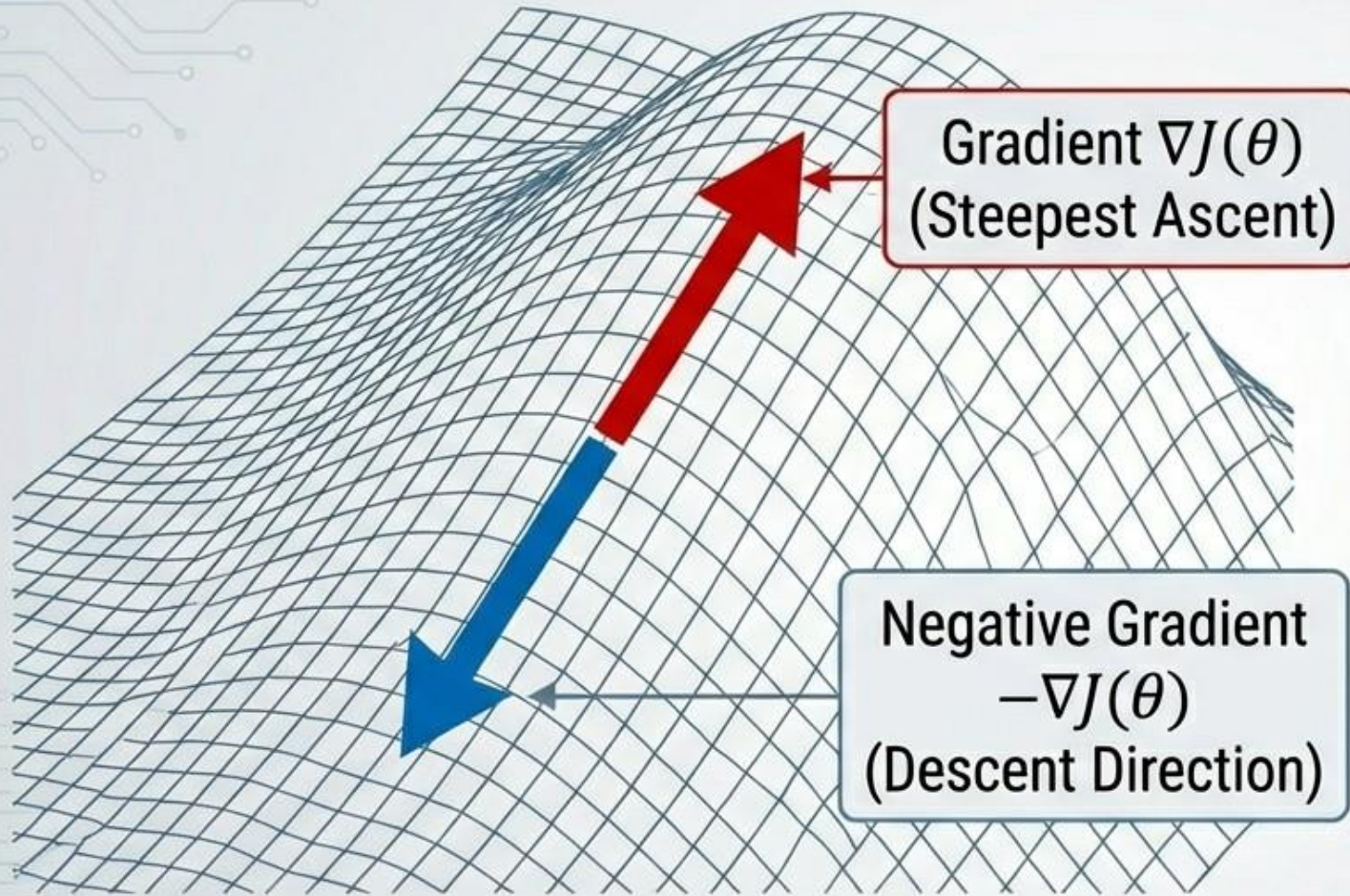
Machine Learning

7

Analogy

- Mountain == Objective function
- Your current position == model parameters (x/y or θ)
- Step size == learning rate (α)
- Lowest point == minima

$\nabla J(\theta)$: The Direction of Steepest Ascent



Partial Derivative ($\partial J / \partial \theta$):
How much error changes if we tweak one parameter.

The Gradient (∇): A vector collecting all partial derivatives. Points to higher error.

The Engineering Decision:
To minimize error, we move opposite the gradient.

The Update Rule

Current Position.
Where the weights are now.

Descent Logic.
We subtract the gradient to go downhill.

Learning Rate.
The step size.
How far do we move?

$$\theta_{new} := \theta_{old} - \alpha \nabla J(\theta)$$

Updated Position.
The weights for the next iteration.

The Gradient.
The direction of steepest ascent.

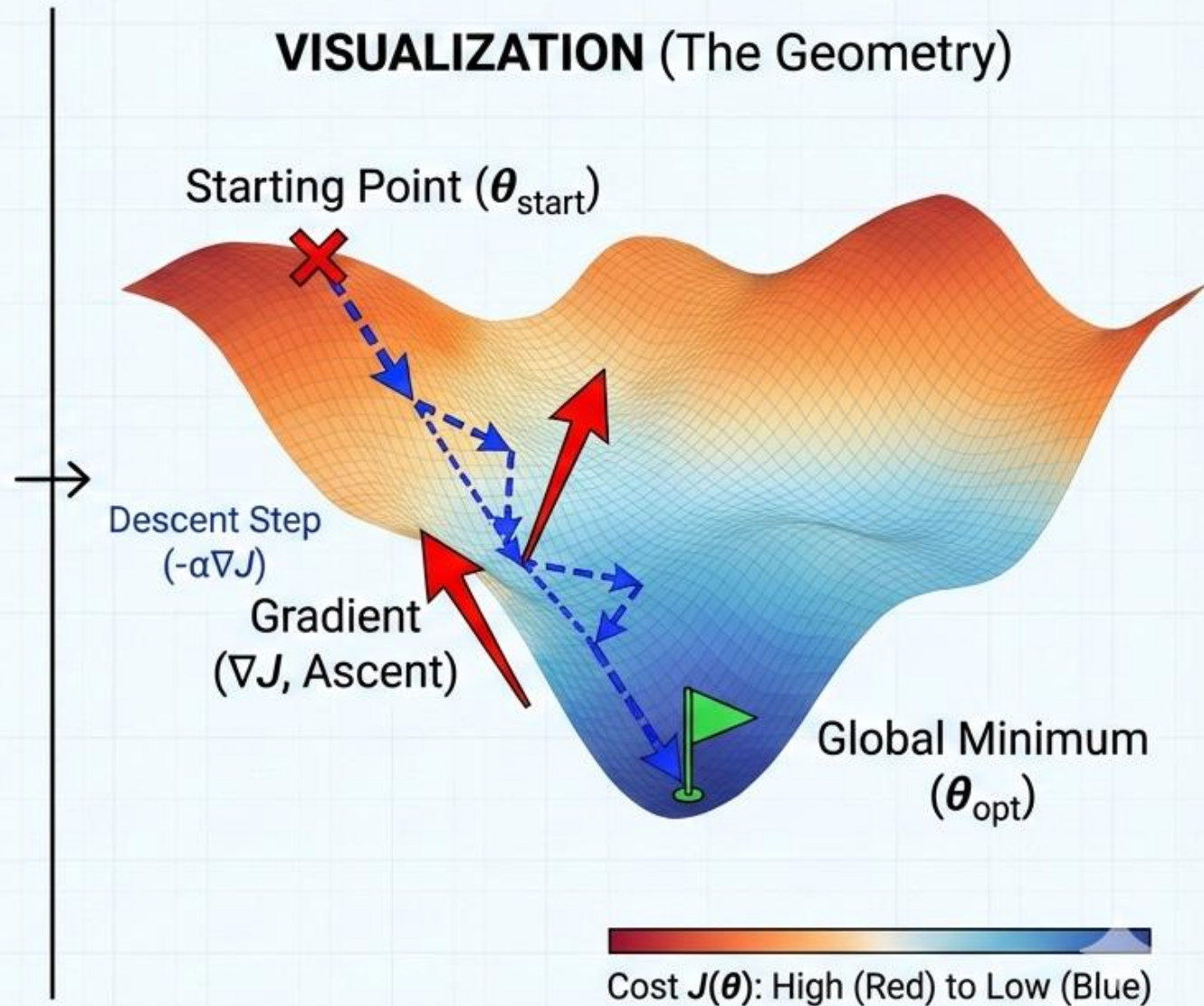
THE GRADIENT DESCENT ALGORITHM: Iterative Optimization Engine

PSEUDOCODE (The Logic)

1. Initialize parameters θ randomly
2. Set learning rate α (step size)
3. REPEAT until convergence:
4. Calculate gradient $\nabla J(\theta)$
 (steepest ascent direction)
5. Update parameters:
 $\theta := \theta - \alpha * \nabla J(\theta)$
6. Check convergence criteria
 (e.g., small change in J)

Goal: Minimize Cost Function $J(\theta)$

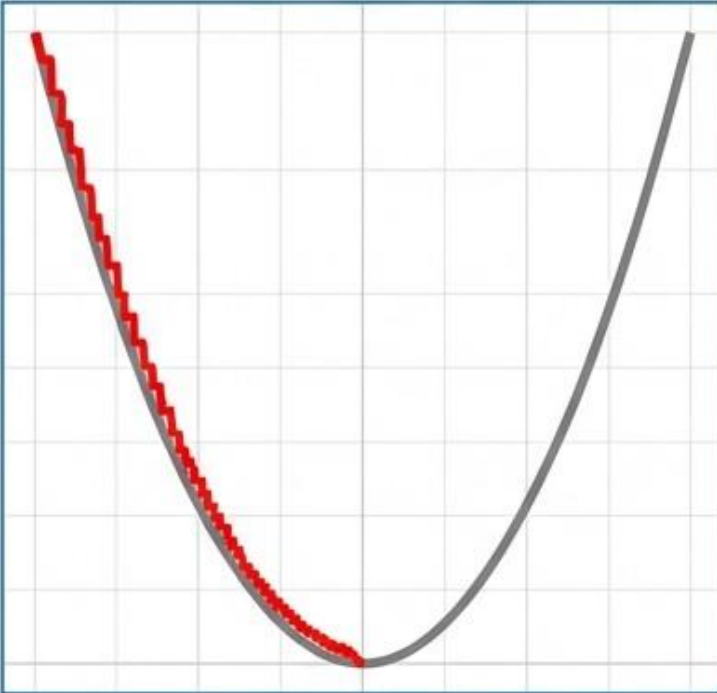
VISUALIZATION (The Geometry)



Gradient Descent in Action

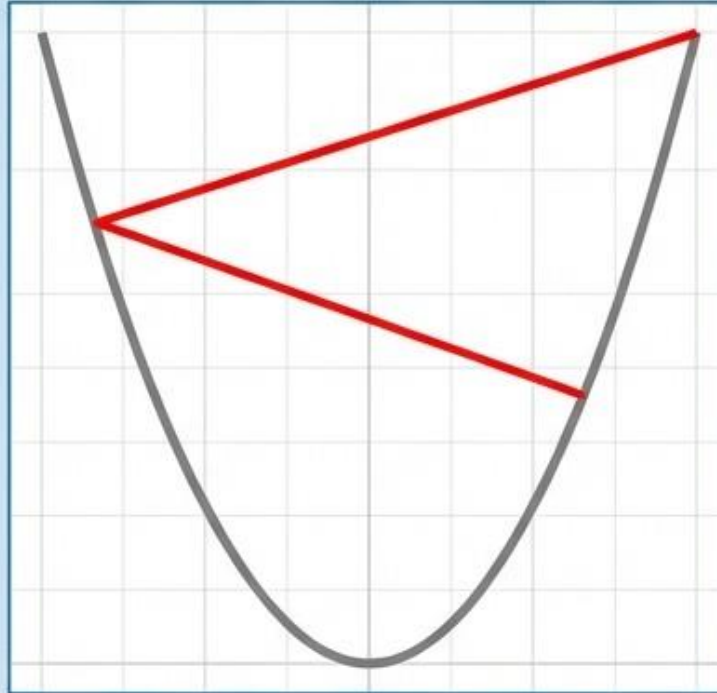
Tuning Alpha (α): The Goldilocks Problem

Low Alpha (Too Small)



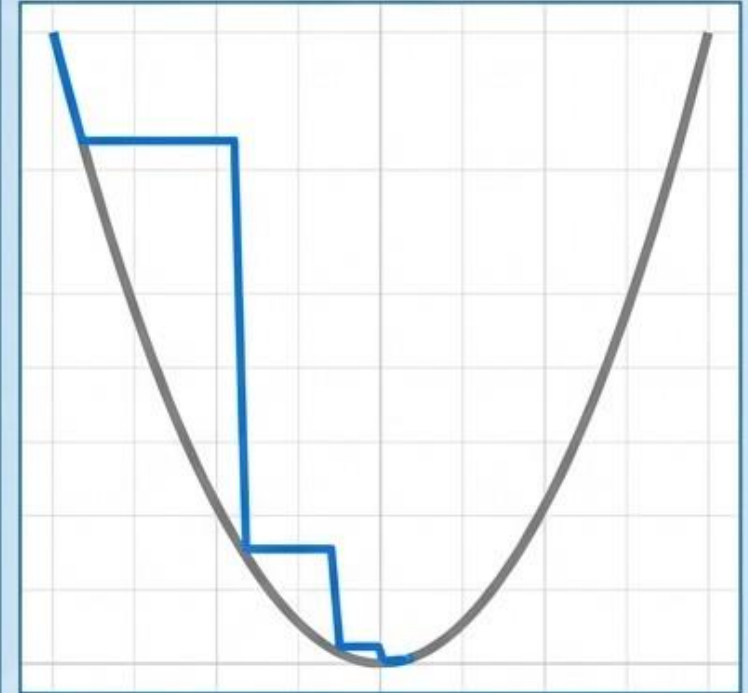
Slow Convergence /
High Compute Cost

High Alpha (Too Large)



Overshooting / Divergence
/ NaN Errors

Optimal Alpha (Just Right)



Rapid Convergence

The Learning Rate is a hyperparameter that acts as a throttle for the descent speed.

Translating Math to Logic (Scalar Implementation)

The Math

Objective: Minimize $f(x) = x^2 + 5$

Derivative: $f'(x) = 2x$

Update Rule: $x := x - \alpha * 2x$

The Python

```
x = 10 # Random initialization
learning_rate = 0.1

for i in range(100):
    gradient = 2 * x # Compute slope
    x = x - (learning_rate * gradient) # Update
print(f'Local Minimum at: {x}')
```



Multivariable Function

- Minimize:

$$f(x, y) = x^2 + y^2$$

- Partial Derivates:

- w.r.t. $x := \frac{\partial f}{\partial x} = 2x$

- w.r.t. $y := \frac{\partial f}{\partial y} = 2y$

- Update Equations:

$$x = x - \alpha \frac{\partial f}{\partial x} = x_{old} - \alpha(2x_{old})$$

$$y = y - \alpha \frac{\partial f}{\partial y} = y_{old} - \alpha(2y_{old})$$

Multivariable Function

- In vectorized form:

$$x_{new} = x_{old} - \alpha(2x_{old})$$

$$y_{new} = y_{old} - \alpha(2y_{old})$$

- $\mathbf{X} = \mathbf{X} - \alpha \nabla f$

- $$\begin{bmatrix} x_{new} \\ y_{new} \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} x_{old} \\ y_{old} \end{bmatrix} - \alpha \begin{bmatrix} 2x_{old} \\ 2y_{old} \end{bmatrix}$$

Lab 1: Optimization from Scratch

Today's Engineering Mission

Scalar Optimization



Minimize $f(x) = x^2 + 5$
using raw Python variables.
Log the path of x .

Vectorized Descent



Minimize a 3D surface
 $f(x, y) = x^2 + y^2$ using
NumPy arrays.
Visualization required.

Failure Mode Analysis



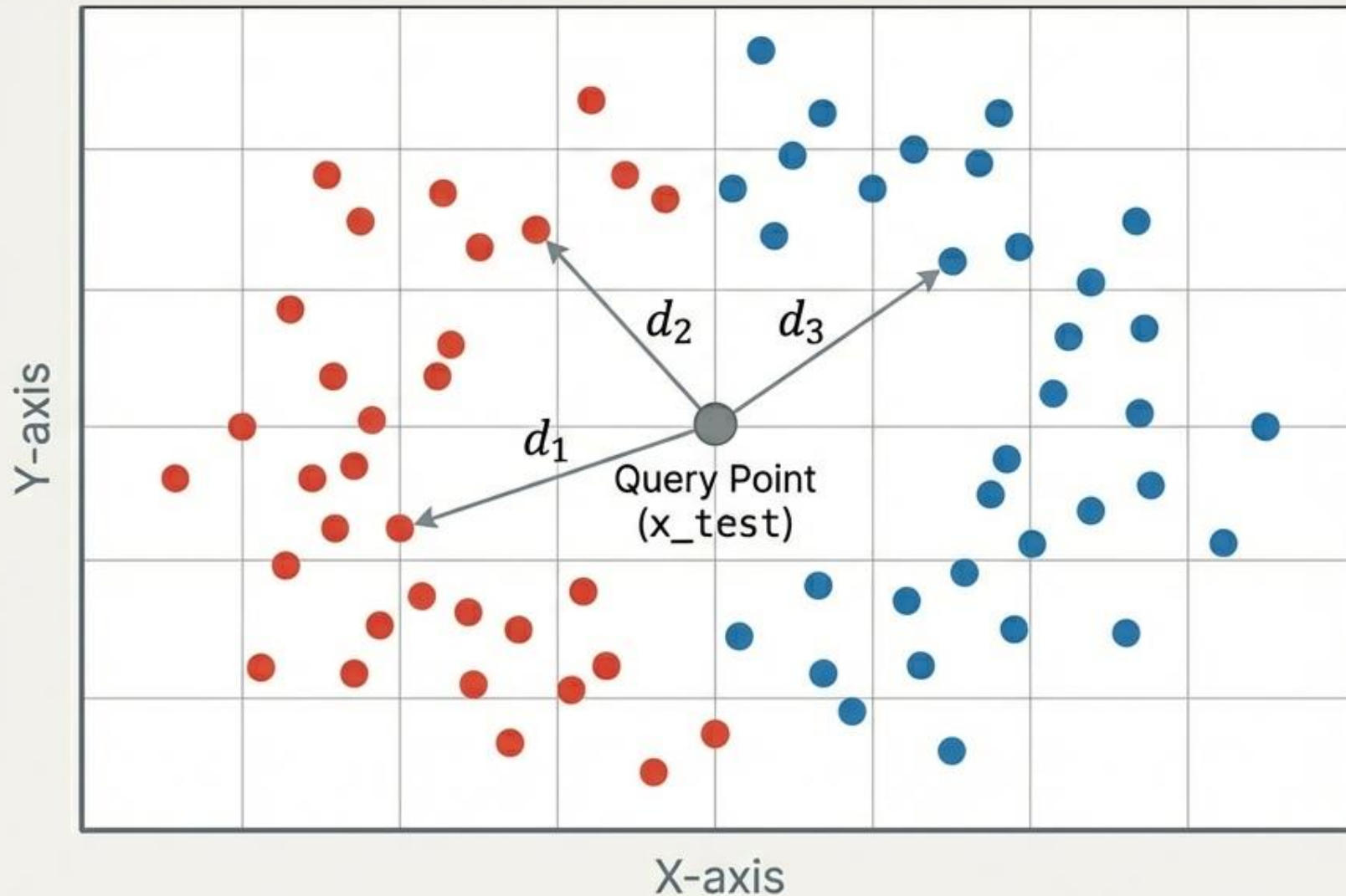
Intentionally set α too high.
Visualize 'Exploding Gradients'
and generate NaN values.

The Math of "Nearness"

- KNN
 - The Simplest Idea in ML.
 - Philosophy: "Tell me who your neighbors are, and I'll tell you who you are."
 - The 3 Steps:
 - Calculate distance → Sort → Vote.

KNN is Geometry, Not 'Learning'

The Lazy Learner Paradigm



No Training Phase

`model.fit()` only stores data in RAM.

Inference Heavy

Prediction requires comparing the query to everyone.

The Intuition

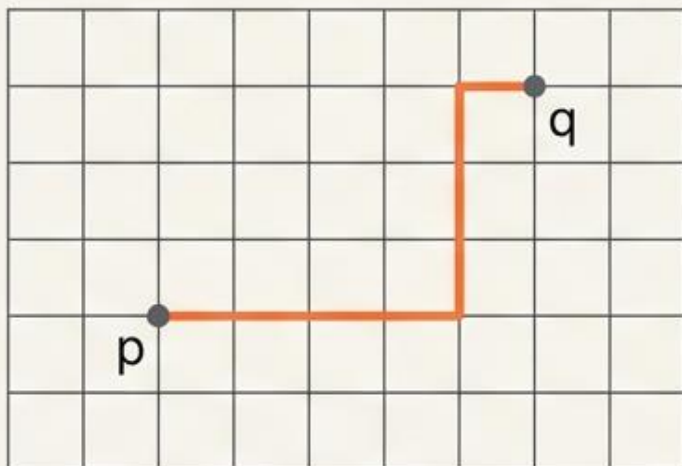
Tell me who your neighbors are, and I'll tell you who you are.

Feature Scaling

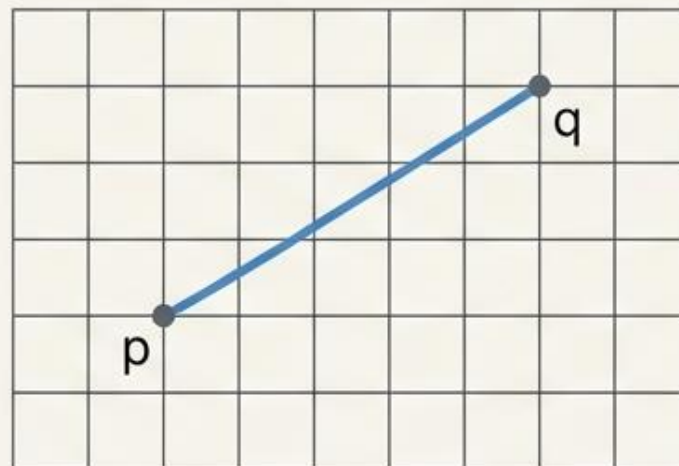
- Why scale matters:
 - If Feature A is "Income" (\$100k) and Feature B is "Age" (25), Income dominates the distance.

Distance is the Core Primitive

Manhattan (L1)



Euclidean (L2)



Translation Layer

Mathematical Notation

$$d(p, q) = \sqrt{\sum (p_i - q_i)^2}$$



NumPy Implementation

```
np.sqrt(np.sum((p - q)**2))
```

Insight: In high dimensions, 1000D images are just vectors. The code is identical.

The Enemy: Python Loops

- **Python Interpretation Overhead**

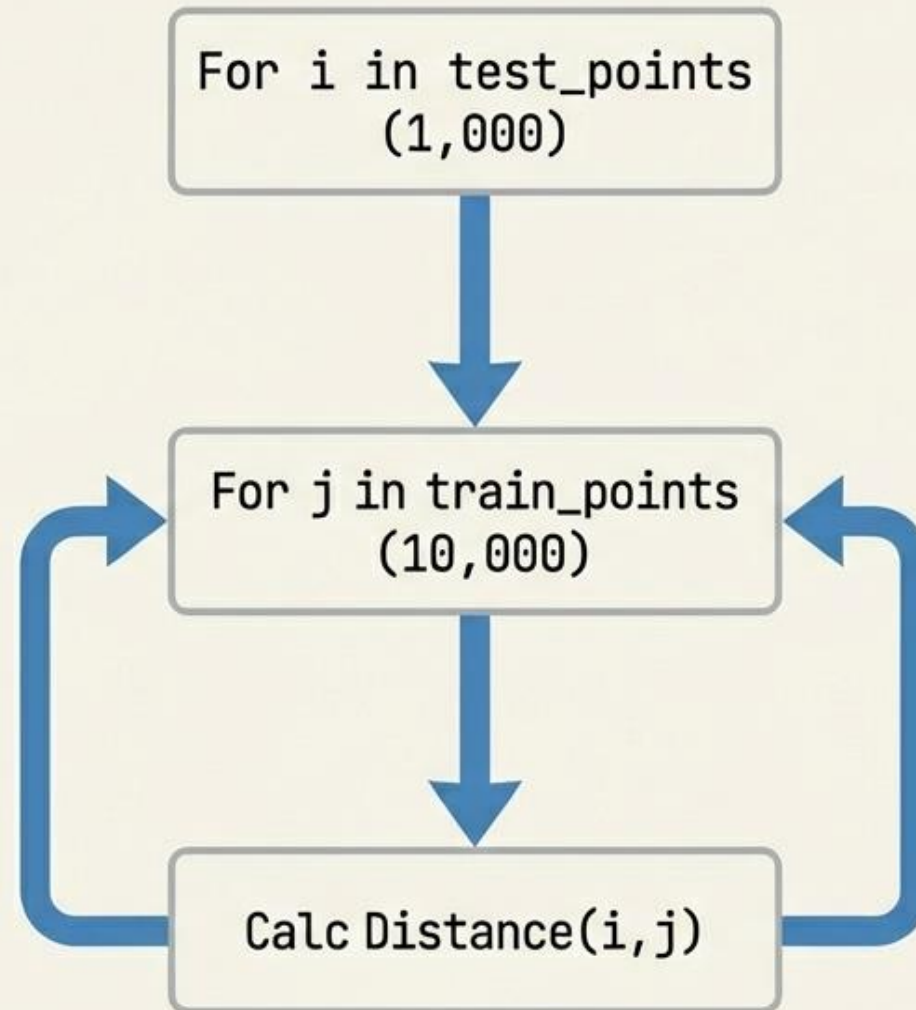
Type-checking occurs at every iteration.

- **The Constraint**

Naive loops are too slow for production systems.

- **Goal**

C-level speed with Python-level syntax.



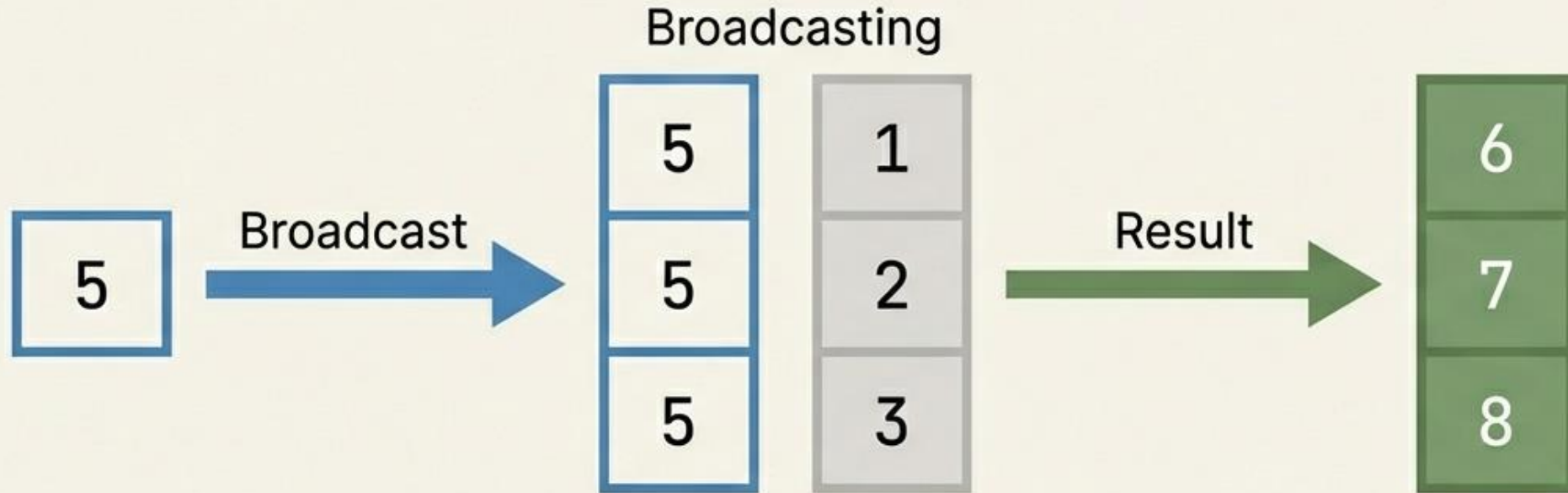
Total Operations:

10,000,000

Calculations

The Solution: NumPy Vectorization

Single Instruction, Multiple Data (SIMD)



Loop (Slow)

`[x + 5 for x in array]`

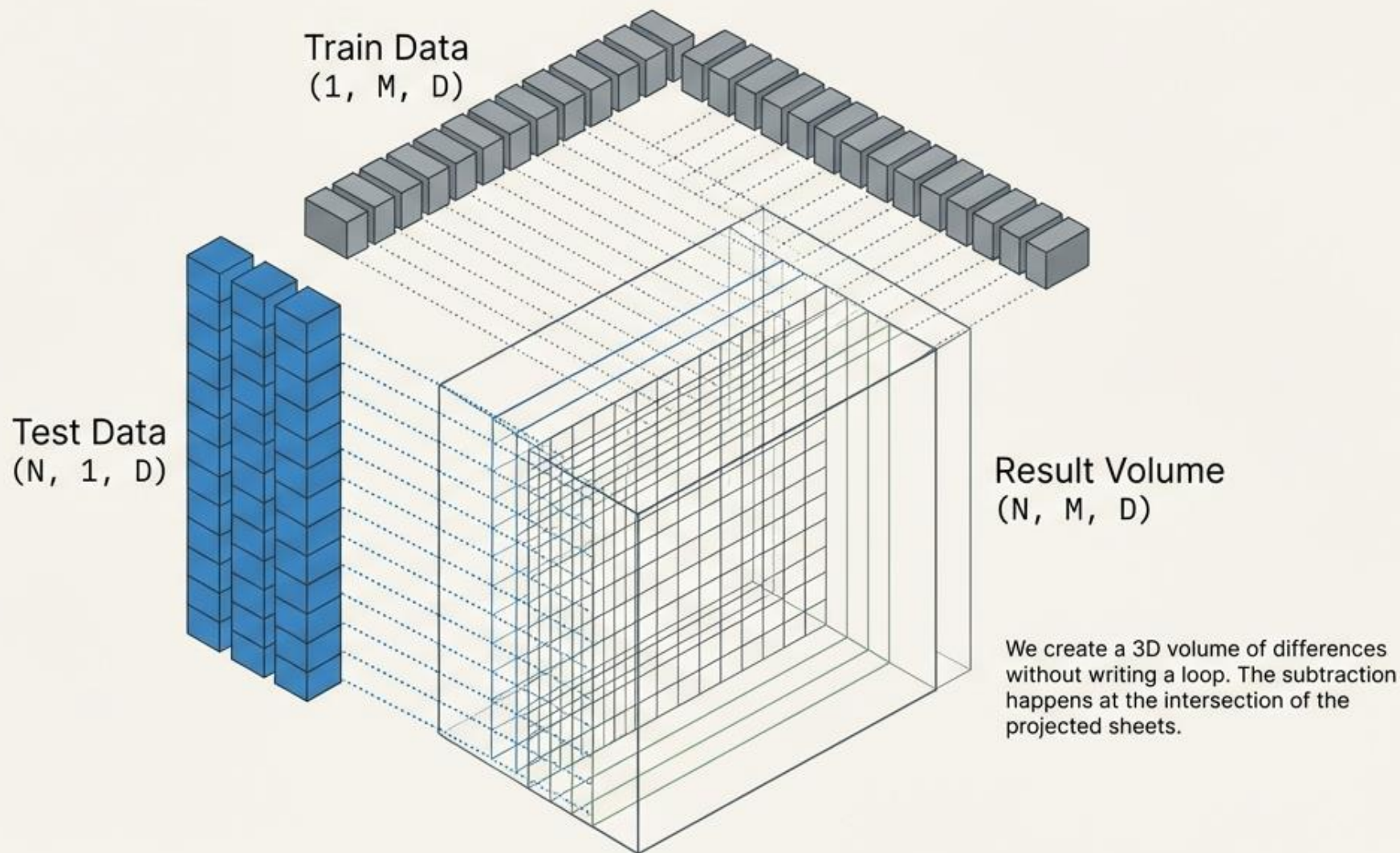
Iterates element-by-element

Vectorized (Fast)

`array + 5`

Offloads to optimized C instructions

Visualizing Broadcasting



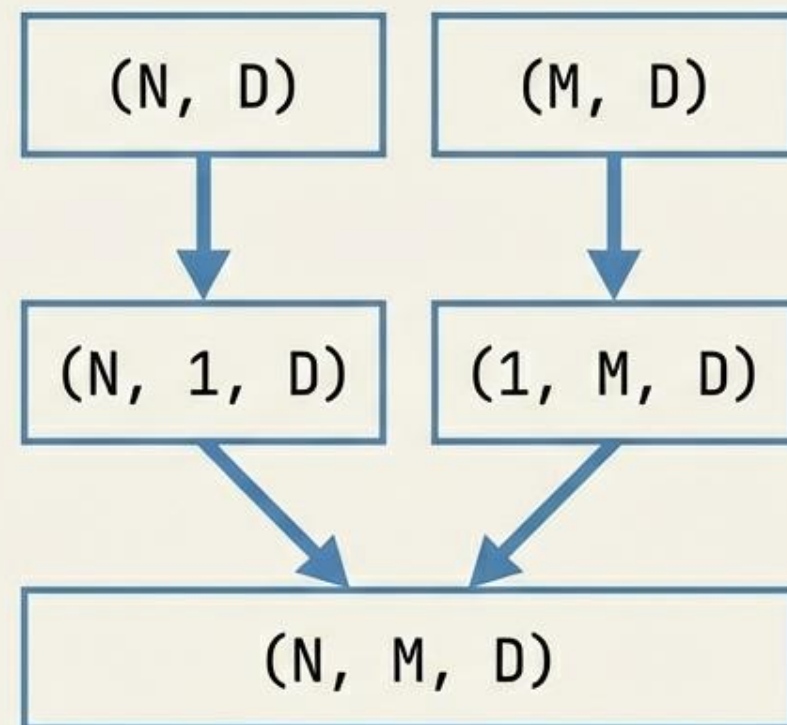
Step 1: The Difference Matrix

```
# X_test: (100, 5) | X_train: (1000, 5)
```

```
# Add new axes to force broadcasting
```

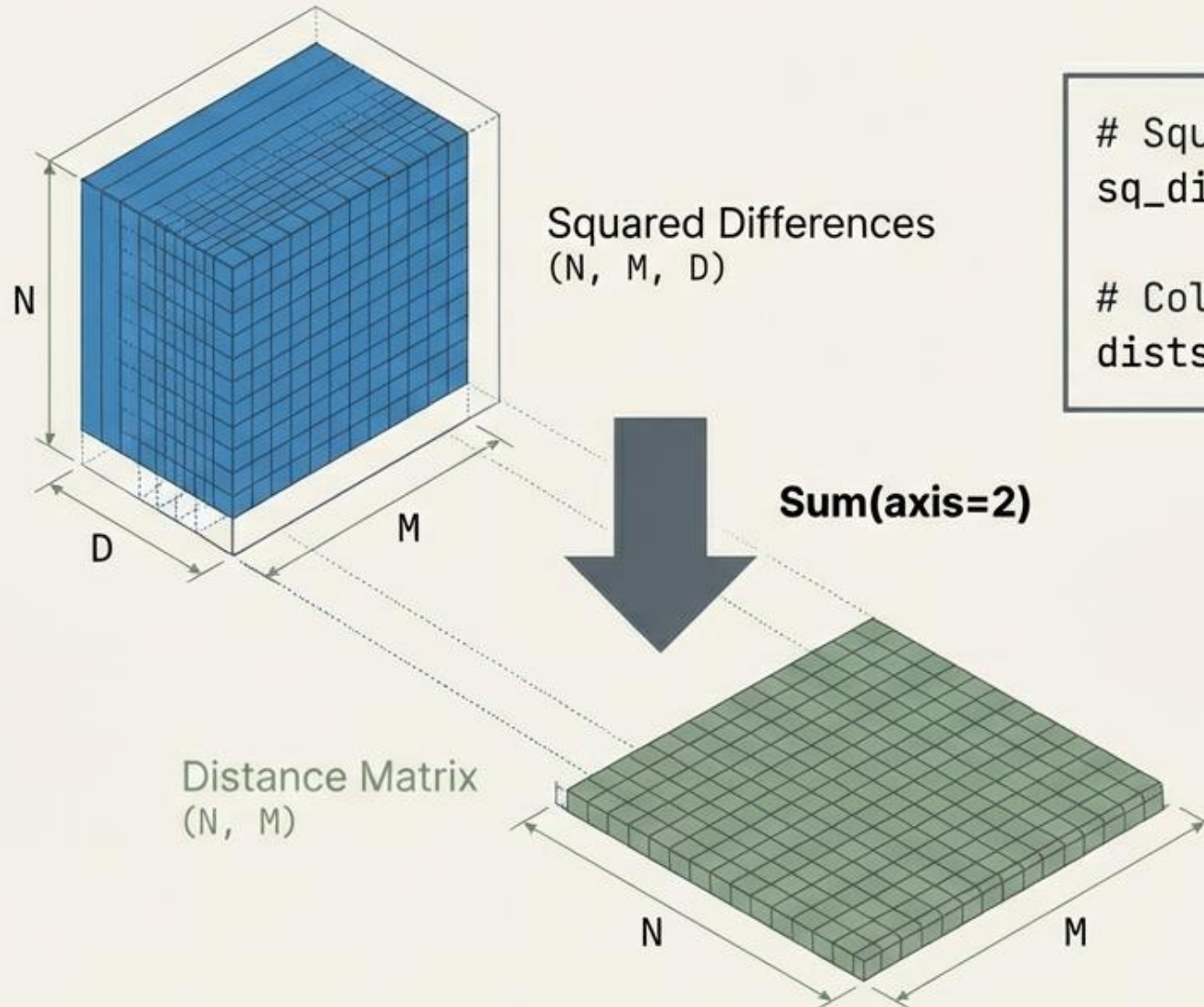
```
diff = X_test[:, np.newaxis, :] -  
       X_train[np.newaxis, :, :]
```

```
# Result Shape: (100, 1000, 5)
```



We now hold the difference for every feature, for every pair of points, in a single tensor.

Step 2: Squash and Sum



```
# Square differences  
sq_diff = diff ** 2
```

```
# Collapse feature dimension  
dists_sq = np.sum(sq_diff, axis=2)
```

`dists_sq[i, j]` is now the
distance between Test Point i
and Train Point j .

Step 3: Efficient Selection

We need the closest K points. We do NOT need to sort the entire list.

Full Sort (np.argsort)



$O(N \log N)$ - Wasteful

Partition (np.argpartition)



$O(N)$ - Efficient

```
k = 3
```

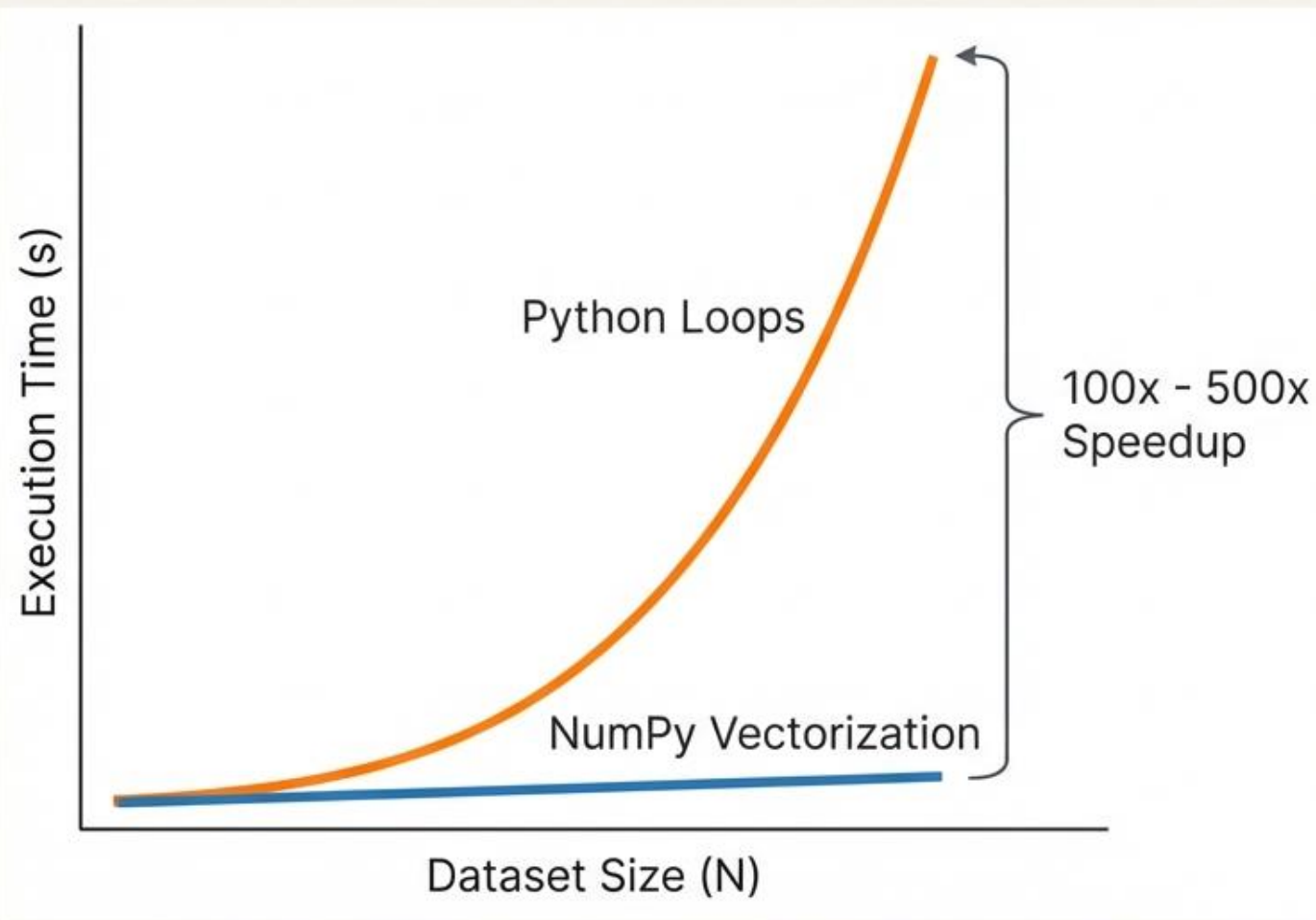
```
# Linear time selection  
nearest_indices = np.argpartition(dists_sq, k, axis=1)[: , :k]
```


Engineering Reality Check

Metric	Complexity	Implication
Training Time	$O(1)$	Instant (Just memory allocation)
Inference Time	$O(N * D)$	Linear - Gets slower as data grows
Space Complexity	$O(N * D)$	Expensive - Must keep all data in RAM

The Scaling Problem: If you have 1 Billion users, you cannot compare a new user to everyone in real-time. Naive KNN fails at scale.

Lab 1: The Speed Test



Goal: Prove the power of vectorization.

Task 1: Implement KNN using pure Python loops.

Task 2: Implement KNN using NumPy broadcasting.

Deliverable: Generate this plot with your own benchmark data.